Complete required programs

All required finite premises and the complete polynomial witness data. Each whole file is printed once, including all checks. GAP finite-presentation algorithms and explicit finite-field certificate verification remain part of the trust boundary.

`required_finite.g`

```
# Complete finite proof for the common projective string-marking theory.
# Reworks the prior six-local/ten-universal census and the
source-correction witnesses.
# GAP's finitely presented group algorithms are retained computational
machinery.
orb:=[]; iso:=fail; vertices:=[];
Fail := function(msg) Error(msg); end;
Need := function(b,msg) if not b then Fail(msg); fi; end;
IP := function(G, rr)
  local ss, hs, a,b;
  ss:=Combinations([1..Length(rr)]);
  hs:=List(ss,s->Subgroup(G,rr{s}));
  for a in [1..Length(ss)] do for b in [1..Length(ss)] do
    if
    Intersection(hs[a],hs[b])<>hs[Position(ss,Intersection(ss[a],ss[b]))]
    then return false; fi;
  od; od; return true;
end;
# The small isomorphisms used in the written applications.
for pp in [[PSL(2,2),SymmetricGroup(3)],[PSL(2,3),AlternatingGroup(4)],
           [PGL(2,3),SymmetricGroup(4)],[PSL(2,5),AlternatingGroup(5)],[P
           GL(2,5),SymmetricGroup(5)]] do
  Need(IsomorphismGroups(pp[1],pp[2])<>fail,"small projective-group
  identification");
od;
# Every ordered string marking; IP is checked only AFTER admission.
for pair in [[SymmetricGroup(4),[[3,3,4],[3,4,3],[4,3,3]]],
             [AlternatingGroup(5),[[3,5,5],[5,3,5],[5,5,3]]]] do
  G:=pair[1]; inv:=Filtered(Elements(G),x->Order(x)=2); markings:=[];
  for a in inv do for c in inv do if a<>c and a*c=c*a then
    for b in inv do rr:=[a,b,c];
      if Length(Set(rr))=3 and Subgroup(G,rr)=G then
        Add(markings,rr);
        Need(Order(a*b)>2 and Order(b*c)>2,"all-sggi adjacent orders");
        Need(IP(G,rr),"all-sggi local intersection property");
      fi;
    od;
  fi; od; od;
  if Size(G)=24 then expected:=72;else expected:=360;fi;
  Need(Length(markings)=expected,"complete all-sggi local census");
  aut:=AutomorphismGroup(G); remaining:=ShallowCopy(markings); types:=[];
  while Length(remaining)>0 do
    rr:=remaining[1]; orb:=Orbit(aut,rr,OnTuples);
    Add(types,[Order(rr[1]*rr[2]),Order(rr[2]*rr[3]),Order(Product(rr))])
    ;
    remaining:=Filtered(remaining,x->not x in orb);
  od;
  Sort(types); Need(types=pair[2],"exceptional rank-three marking
  census");
  F3:=FreeGroup("x","y","z"); z3:=GeneratorsOfGroup(F3);
```

```
  for typ in types do
    R3:=Concatenation(List(z3,a->a^2),[(z3[1]*z3[3])^2,
      (z3[1]*z3[2])^typ[1],(z3[2]*z3[3])^typ[2],Product(z3)^typ[3]]);
    Need(Size(F3/R3)=Size(G),"complete rank-three presentation");
  od;
  Print("RANK3 ",Size(G)," MARKINGS ",Length(markings)," TYPES
  ",types,"\n");
od;
# [facet m,n,k; vertex m,n,k; universal order; string IP; face counts]
rows := [
 [[5,5,3],[5,5,3],1,false,[]],
 [[5,5,3],[5,3,5],1,false,[]],
 [[5,3,5],[3,5,5],3420,true,[57,171,171,57]],
 [[5,3,5],[3,4,3],60,false,[]],
 [[5,3,5],[3,3,4],1,false,[]],
 [[4,3,3],[3,4,3],96,true,[4,6,6,4]],
 [[4,3,3],[3,3,4],24,false,[]],
 [[3,5,5],[5,3,5],660,true,[11,55,55,11]],
 [[3,4,3],[4,3,3],1,false,[]],
 [[3,3,4],[3,3,4],120,true,[5,10,10,5]] ];
localtypes:=[[3,3,4],[3,4,3],[4,3,3],[3,5,5],[5,3,5],[5,5,3]];
allpairs:=Filtered(Cartesian(localtypes,localtypes),p->p[1][2]=p[2][1]);
revtype:=t->[t[2],t[1],t[3]];
revpair:=p->[revtype(p[2]),revtype(p[1])];
covered:=Set(Concatenation(List(rows,r->[r[1],r[2]]),List(rows,r->revpair
([r[1],r[2]]))));
Need(Length(allpairs)=14 and Set(allpairs)=covered,"exhaustive compatible
local pairs");
Need(Length(Filtered(allpairs,p->p=revpair(p)))=6,"six self-dual local
pairs");
F:=FreeGroup("r0","r1","r2","r3"); x:=GeneratorsOfGroup(F);
for n in [1..Length(rows)] do
  row:=rows[n]; f:=row[1]; v:=row[2]; Need(f[2]=v[1],"shared edge
  label");
  rel:=List(x,a->a^2);
  Append(rel,[(x[1]*x[2])^f[1],(x[2]*x[3])^f[2],(x[3]*x[4])^v[2],
              (x[1]*x[3])^2,(x[1]*x[4])^2,(x[2]*x[4])^2,
              (x[1]*x[2]*x[3])^f[3],(x[2]*x[3]*x[4])^v[3]]);
  U:=F/rel; Need(Size(U)=row[3],"fp universal order"); if row[3]=1 then
  G:=Group(()); rr:=[(),(),(),()]; else iso:=IsomorphismPermGroup(U);
  Need(iso<>fail,"fp permutation isomorphism"); G:=Image(iso);
  rr:=List(GeneratorsOfGroup(U),a->Image(iso,a)); fi;
  Need(Size(G)=row[3],Concatenation("universal order row ",String(n)));
  good:=ForAll(rr,a->Order(a)=2) and Length(Set(rr))=4 and IP(G,rr);
  Need(good=row[4],"rank-four intersection property");
  if good then
    faces:=List([1..4],i->Index(G,Subgroup(G,rr{Difference([1..4],[i])}))
    );
    Need(faces=row[5],"face counts");
  else faces:=[]; fi;
  if n in [3,8,10] then
    if n=3 then target:=PSL(2,19); elif n=8 then target:=PSL(2,11); else
    target:=SymmetricGroup(5); fi;
    Need(IsomorphismGroups(G,target)<>fail,"universal group
    identification");
    dual:=GroupHomomorphismByImages(G,G,rr,Reversed(rr));
    Need(dual<>fail and IsBijective(dual),"self-duality");
  fi;
```

```
  if n in [4,7] then
    if n=4 then target:=AlternatingGroup(5); else
    target:=SymmetricGroup(4); fi;
    Need(IsomorphismGroups(G,target)<>fail,"collapsed group
    identification");
    Need(Subgroup(G,rr{[1,2,3]})=G or
    Subgroup(G,rr{[2,3,4]})=G,"collapsed full facet or vertex figure");
  fi;
  if n=6 then
    V:=FittingSubgroup(G); Need(Size(V)=16 and
    IsElementaryAbelian(V),"row-six elementary normal subgroup");
    Need(IsomorphismGroups(FactorGroup(G,V),SymmetricGroup(3))<>fail,"row
    -six quotient");
    Need(Length(ComplementClassesRepresentatives(G,V))>0,"row-six
    splitting");
  fi;
  if n=8 then
    vertices:=RightCosets(G,Subgroup(G,rr{[2,3,4]}));
    edges:=RightCosets(G,Subgroup(G,rr{[1,3,4]}));
    ends:=List(edges,e->Filtered([1..11],i->not
    IsEmpty(Intersection(Set(e),Set(vertices[i])))));
    Need(ForAll(ends,e->Length(e)=2) and Length(Set(ends))=55,"11-cell
    complete edge graph");
  fi;
  if n=6 then
    Need(Order(rr[1]*rr[2])=4 and Order(rr[3]*rr[4])=4,"order96 outer
    labels are4 not Petrie3");
    for indices in [[1,2,3],[2,3,4]] do
      H:=Subgroup(G,rr{indices});
      Need(Size(H)=24 and H<>G and
      IsomorphismGroups(H,SymmetricGroup(4))<>fail,"proper S4 ends in
      parity sharpness example");
    od;
  fi;
  Print("ROW ",n," ORDER ",Size(G)," IP ",good," FACES ",faces,"\n");
od;

StringMarking:=function(rr)
 local i,j;
 if Length(Set(rr))<>Length(rr) or not ForAll(rr,x->Order(x)=2) then
 return false;fi;
 for i in [1..Length(rr)] do for j in [i+1..Length(rr)] do
 if j>i+1 and rr[i]*rr[j]<>rr[j]*rr[i] then return false;fi;
 od;od;return true;
end;
 # Exact counterexample to literal source Lemma3.3 maximality.
 G:=AlternatingGroup(5);z:=(1,2)(3,4);H:=Stabilizer(G,5);cs:=Centralizer(
 G,z);
 Need(Size(cs)=4 and Size(H)=12 and IsSubgroup(H,cs) and
 H<>G,"centralizer maximality counterexample");
 Print("SOURCE_L33_COUNTEREXAMPLE PSL5 CENTRALIZER4 CONTAINED_A4_12\n");
 # The smallest omitted S4 example is literal, avoiding a subgroup
 search.
 rr:=[(1,2)(3,24)(4,23)(5,22)(6,21)(7,20)(8,19)(9,18)(10,17)(11,16)(12,15
 )(13,14),
 (1,5)(2,7)(3,21)(4,11)(6,17)(8,23)(9,13)(10,24)(12,14)(15,18)(16,19)(20,
 22),
```

```
 (1,4)(2,23)(3,9)(5,13)(6,16)(7,10)(8,15)(11,21)(12,19)(14,22)(17,20)(18,
 24),
 (1,10)(2,17)(3,22)(4,18)(5,24)(6,7)(8,13)(9,23)(11,15)(12,16)(14,19)(20,
 21)];
 G:=PSL(2,23);Need(ForAll(rr,x->x in G) and Group(rr)=G and
 StringMarking(rr),"literal PSL23 marking");
 H:=Subgroup(G,rr{[1..3]});Need(Size(H)=24 and
 IsomorphismGroups(H,SymmetricGroup(4))<>fail,"PSL23 S4 end");
 Need(Subgroup(G,rr{[2..4]})=G and Index(G,H)=253,"omitted253facet
 count");
 Need(List([1..3],i->Order(rr[i]*rr[i+1]))=[3,4,12],"literal
 omitted-family type");
 Print("OMITTED_S4 PSL23 ORDER6072 FACETS253 VERTICES1 TYPE3,4,12
 PASS\n");
 # A literal self-dual Class1 marking and its actual degree14 conjugator.
 rr:=[(3,9)(4,10)(5,11)(6,12)(7,13)(8,14),(1,4)(2,14)(3,9)(5,13)(6,11)(7,
 12),
 (1,2)(3,7)(4,6)(8,14)(9,13)(10,12),(1,2)(4,14)(5,13)(6,12)(7,11)(8,10)];
 G:=PSL(2,13);dual:=(1,3)(2,9)(4,7)(5,8)(6,12)(10,11)(13,14);
 Need(ForAll(rr,x->x in G) and Group(rr)=G and StringMarking(rr),"literal
 selfdual marking");
 Need(Subgroup(G,rr{[1..3]})=G and Subgroup(G,rr{[2..4]})=G,"selfdual
 Class1 ends");
 Need(List([1..3],i->Order(rr[i]*rr[i+1]))=[3,7,3] and
 List(rr,x->x^dual)=Reversed(rr),"literal selfdual reversal");
 Need(List(rr,x->x^(dual*(1,2)))<>Reversed(rr),"altered conjugator
 rejected");
 Print("SELFDUAL_CLASS1 PSL13 ORDER1092 TYPE3,7,3 CONJUGATOR_PASS
 ALTERED_REJECTED\n");

Print("TOPIC_E_REQUIRED_FINITE_PASS\n");
QUIT;
```

exceptional_amalgams.g

```
# Exact finite premises for the exceptional rotation-end classification.
# Uses GAP's finite-presentation and subgroup algorithms. No random
search.
Need:=function(b,m) if not b then Error(m);fi;end;
Base:=function(typ)
 local F,x,rel;
 F:=FreeGroup("a","b","c");x:=GeneratorsOfGroup(F);
 rel:=[x[1]^typ[1],x[2]^typ[2],x[3]^typ[3],(x[1]*x[2])^2,(x[2]*x[3])^2,Pr
 oduct(x)^2];
 return [F,x,rel];
end;
Run:=function()
 local G,ps,a,b,c,au,rem,orb,types,pair,typ,base,F,x,rel,t1,t2,t3,t4,k,j,
 i,l,ord,orders,expected,rels,U,iso,P,cf,n,ss,hs,As,chosen,rt,lt;
 for G in [AlternatingGroup(4),SymmetricGroup(4),AlternatingGroup(5)] do
  ps:=[];
  for a in Elements(G) do for b in Elements(G) do
   if Order(a)>2 and Order(b)>2 and Order(a*b)=2 and Group(a,b)=G and
   Size(Intersection(Group(a),Group(b)))=1 then Add(ps,[a,b]);fi;
  od;od;
  au:=AutomorphismGroup(G);rem:=ShallowCopy(ps);types:=[];
  while Length(rem)>0 do
```

```
  pair:=rem[1];orb:=Orbit(au,pair,OnTuples);Add(types,List(pair,Order));
  rem:=Filtered(rem,z->not z in orb);
 od;
 if Size(G)=12 then expected:=[[3,3]];n:=24;elif Size(G)=24 then
 expected:=[[3,4],[4,3]];n:=48;else
 expected:=[[3,5],[5,3],[5,5]];n:=360;fi;
 Need(Set(types)=Set(expected) and Length(ps)=n,"complete local pair
 census");
 if Size(G)=60 then
  for pair in ps do
   a:=pair[1];b:=pair[2];
   if List(pair,Order)=[5,5] then
   Need(a*b^2*a^3*b^2*a^2=One(G),"complete five-five local relator");fi;
   if Order(b) in [3,5] and Order(a)<>Order(b) then
    t1:=a*b^2*a^2*b^2*a^2;
    Need(Order(t1)=2 and b^t1=b^-1,"literal dihedral normalizer word");
    Need(Normalizer(G,Group(b))=Group(b,t1),"complete cyclic
    normalizer");
   fi;
   if List(pair,Order)=[5,3] then
    t1:=a^2*b^-1*a;t2:=a*b^-1*a^2;
    Need(Order(t1*b)=2 and Order(t2*b)=2,"A4 word involutions");
    As:=Filtered(AllSubgroups(G),H->Size(H)=12 and
    IsSubgroup(H,Group(b)));
    Need(Length(As)=2 and Set([Group(t1,b),Group(t2,b)])=Set(As),"both
    A4 overgroups");
    for chosen in [t1,t2] do Need(Set(List([0..2],k->(chosen*b)^(b^k)))=
    Set(Filtered(Elements(Group(chosen,b)),z->Order(z)=2)),"three A4
    involutions by beta");od;
   fi;
  od;
 fi;
 Print("LOCAL_PAIRS ",Size(G)," ",Length(ps)," ",types," PASS\n");
od;
for pair in [[[3,3,3],60],[[3,3,4],192],[[3,4,3],576],[[3,3,5],7200]] do
 base:=Base(pair[1]);U:=base[1]/base[3];Need(Size(U)=pair[2],"spherical
 rotation order");
 iso:=IsomorphismPermGroup(U);P:=Image(iso);cf:=List(CompositionSeries(P
 ),Size);
 Need(ForAll([1..Length(cf)-1],k->cf[k]/cf[k+1] in [2,3,60]),"spherical
 simple factors");
 Print("SPHERICAL ",pair[1]," ORDER ",Size(P)," SERIES ",cf,"\n");
od;
# Source exclusion of the remaining five-five types. Full presentation,
not quotient sampling.
for typ in [[5,5,3],[5,5,5]] do
 base:=Base(typ);F:=base[1];x:=base[2];rel:=ShallowCopy(base[3]);a:=x[1]
 ;b:=x[2];c:=x[3];
 Add(rel,a*b^2*a^3*b^2*a^2);
 if typ[3]=5 then Add(rel,b*c^2*b^3*c^2*b^2);fi;
 U:=F/rel;Need(Size(U)=7200,"five-five universal
 order");iso:=IsomorphismPermGroup(U);P:=Image(iso);
 cf:=List(CompositionSeries(P),Size);Need(Set(Factors(Size(P)))=[2,3,5],
 "five-five order primes");
 Need(ForAll([1..Length(cf)-1],k->cf[k]/cf[k+1] in [2,60]),"five-five
 simple factors C2,A5");
 Print("FIVE_FIVE ",typ," ORDER ",Size(P)," SERIES ",cf,"\n");
od;
```

```
 for typ in [[5,3,4],[5,3,5],[3,5,3]] do
  base:=Base(typ);F:=base[1];x:=base[2];a:=x[1];b:=x[2];c:=x[3];
  t1:=a*b^2*a^2*b^2*a^2;
  if typ[3]=4 then t2:=c*b^2*c^2;expected:=[60,1,1];
  elif typ[2]=3 then t2:=c*b^2*c^2*b^2*c^2;expected:=[3420,60,60];
  else t2:=c*b^2*c^2*b^2*c^2;expected:=[660,1,360,360,1];fi;
  orders:=[];
  for k in [1..typ[2]] do
   rel:=Concatenation(base[3],[t2/(t1*b^k)]);U:=F/rel;ord:=Size(U);Add(or
   ders,ord);Need(ord=expected[k],"dihedral collision universal order");
   if ord>2 then
    iso:=IsomorphismPermGroup(U);P:=Image(iso);
    if ord=60 then G:=AlternatingGroup(5);elif ord=360 then
    G:=PSL(2,9);elif ord=660 then G:=PSL(2,11);else G:=PSL(2,19);fi;
    Need(IsomorphismGroups(P,G)<>fail,"dihedral collision group
    identification");
   fi;
  od;
  Print("DIHEDRAL_COLLISION ",typ," ORDERS ",orders,"\n");
  if typ[2]=3 then
   t1:=a^2*b^-1*a;t2:=a*b^-1*a^2;
   if typ[3]=4 then rt:=[b^2*c^2];expected:=[2,2,2,2,2,2];else rt:=[c^2*b
   ^-1*c,c*b^-1*c^2];expected:=[1,1,1,1,60,60,60,1,60,1,1,1];fi;
   orders:=[];
   for lt in [t1,t2] do for t3 in rt do for j in [0..2] do
    rel:=Concatenation(base[3],[lt^(b^j)/t3]);U:=F/rel;ord:=Size(U);Add(o
    rders,ord);Need(ord=expected[Length(orders)],"A4 collision universal
    order");
    if ord=60 then iso:=IsomorphismPermGroup(U);Need(IsomorphismGroups(Im
    age(iso),AlternatingGroup(5))<>fail,"A4 collision quotient A5");fi;
   od;od;od;
   Need(orders=expected,"complete A4 collision
   table");Print("A4_COLLISION ",typ," ORDERS ",orders,"\n");
  fi;
 od;
 G:=AlternatingGroup(5);a:=(1,2,3,4,5);b:=(2,5,3);
 Need(Order(a*b)=2,"fixed A5 seed");
 hs:=Filtered(AllSubgroups(G),H->IsSubgroup(H,Group(b)));
 Need(Set(List(hs,Size))=[3,6,12,60],"all A5 C3 overgroup sizes");
 ps:=Filtered(Elements(G),c->Order(c)=5 and Order(b*c)=2 and
 Order(a*b*c)=2);
 Need(Length(ps)=2 and ForAll(ps,c->not IsConjugate(G,a,c)),"mixed-class
 extensions lie in A5");
 a:=(3,4,5);b:=(1,3,5,2,4);
 hs:=Filtered(AllSubgroups(G),H->IsSubgroup(H,Group(b)));
 Need(Set(List(hs,Size))=[5,10,60],"all A5 C5 overgroup sizes");
 Need(IsEmpty(Filtered(Elements(G),c->Order(c)=3 and Order(b*c)=2 and
 Order(a*b*c)=2)),"no 353 extension inside A5");
 G:=SymmetricGroup(4);b:=(2,3,4);c:=(1,2,4,3);
 Need(Order(b*c)=2 and Group(b,c)=G,"fixed S4 seed");
 t2:=c*b^2*c^2;t3:=b^2*c^2;
 Need(Order(t2)=2 and b^t2=b^-1 and
 Normalizer(G,Group(b))=Group(b,t2),"S4 normalizer word");
 Need(Order(t3*b)=2 and Group(b,t3)=AlternatingGroup(4),"S4 A4 word");
 Print("EXCEPTIONAL_AMALGAMS_PASS\n");
end;
Run();QUIT;
```

exceptional_small.g

```
# Required exceptional-prime checks for the trace-quadratic
classification.
Need:=function(b,m) if not b then Error(m);fi;end;
ProjectivePerm:=function(M,F)
 local pts,n,im,v,w,pos;
 pts:=Elements(F);n:=Length(pts);im:=[];
 for pos in [1..n+1] do
  if pos<=n then v:=[pts[pos],One(F)];else v:=[One(F),Zero(F)];fi;
  w:=v*M;
  if IsZero(w[2]) then Add(im,n+1);else
  Add(im,Position(pts,w[1]/w[2]));fi;
 od;return PermList(im);
end;
Run:=function()
 local p,F,o,z,root,ts,typ,tr,aa,bb,cc,d,U,V,u,w,A,B,C,J,delta,rt,R,S,rr,
 H,K,G,rows,one,cs,good,rev,X;
 for p in [11,19] do
  F:=GF(p);o:=One(F);z:=Zero(F);J:=[[z,o],[-o,z]];
  ts:=List(Filtered(Elements(F),v->v^2=5*o),v->(-o+v)/2);
  for typ in [[5,3,5],[3,5,3]] do
   rows:=[];
   for tr in ts do
    if typ=[5,3,5] then aa:=tr;bb:=o;cc:=tr;else aa:=o;bb:=tr;cc:=o;fi;
    d:=aa^2+bb^2-4*o;Need(d<>z,"irreducible seed");
    one:=First(Cartesian(Elements(F),Elements(F)),v->v[1]^2+v[2]^2=d);U:=
    one[1];V:=one[2];
    u:=(U+aa)/2;w:=(V-bb)/2;A:=[[u,w+bb],[w,aa-u]];B:=A^-1*J;
    delta:=aa^2*cc^2-4*(aa^2+bb^2+cc^2-4*o);
    for rt in Filtered(Elements(F),v->v^2=delta) do
     R:=(-bb*cc*V+rt*U)/d;S:=(bb*cc*U+rt*V)/d;C:=[[(R+cc)/2,S/2],[S/2,(cc
     -R)/2]];
     Need(List([A,B,C],DeterminantMat)=[o,o,o],"determinants");
     Need(List([A,B,C],TraceMat)=[aa,bb,cc] and
     ForAll([A*B,B*C,A*B*C],M->TraceMat(M)=z),"all trace equations");
     rr:=List([A,B,C],M->ProjectivePerm(M,F));Need(List(rr,Order)=typ and
     List([rr[1]*rr[2],rr[2]*rr[3],Product(rr)],Order)=[2,2,2],"exact
     relations");
     H:=Group(rr{[1,2]});K:=Group(rr{[2,3]});G:=Group(rr);
     Need(Size(H)=60 and Size(K)=60 and Size(G)=p*(p^2-1)/2,"full
     groups");
     good:=Size(Intersection(H,K))=typ[2];
     Need(Size(Intersection(Group(rr[1]),Group(rr[2])))=1 and
     Size(Intersection(Group(rr[2]),Group(rr[3])))=1,"cyclic IP");
     if rt<>z then Need(good,"every nonzero discriminant extension is
     polytopal");fi;
     if rt=z then
      X:=C-(cc/2)*IdentityMat(2,F);rev:=ProjectivePerm(X,F);
      Need(rr[1]^rev=rr[1]^-1 and rr[2]^rev=rr[1]^2*rr[2] and
      rr[3]^rev=rr[3],"zero discriminant actual reversal");
     fi;
     Add(rows,[Int(tr),Int(rt),Size(Intersection(H,K)),good]);
    od;
   od;
   good:=Length(Filtered(rows,r->r[2]<>0 and r[4]));
   if p=11 and typ=[3,5,3] then Need(good=0,"no chiral PSL11 353");fi;
   if p=19 and typ=[5,3,5] then Need(good=2,"exactly two chiral PSL19
   535");fi;
```

```
   Print("SMALL ",p," TYPE ",typ," ROWS ",rows," CHIRAL ",good,"\n");
  od;
 od;
 Print("EXCEPTIONAL_SMALL_PASS\n");
end;
Run();QUIT;
```

small_rotation_exclusions.g

```
# Complete small-group exclusion, modulo actual automorphisms of each
group.
Need:=function(b,m)if not b then Error(m);fi;end;
Run:=function()
 local n,G,au,els,inv,reps,a,b,c,rr,H,K,st,bs,pairs,triples,poly,rev,nonc
 hiral,images,perms,r,ts;
 for n in [5,6] do
  G:=AlternatingGroup(n);au:=AutomorphismGroup(G);els:=Elements(G);inv:=F
  iltered(els,x->Order(x)=2);
  reps:=List(Orbits(au,els),x->x[1]);pairs:=0;triples:=0;poly:=0;nonchira
  l:=0;
  for a in Filtered(reps,x->Order(x)>2) do
   st:=Stabilizer(au,a);bs:=Filtered(els,b->Order(b)>2 and Order(a*b)=2
   and Size(Intersection(Group(a),Group(b)))=1);
   for b in List(Orbits(st,bs),x->x[1]) do
    H:=Group(a,b);if H=G then continue;fi;pairs:=pairs+1;r:=a*b;
    for ts in inv do
     c:=r*ts;
     if Order(c)<=2 or Order(b*c)<>2 then continue;fi;
     K:=Group(b,c);if K=G or Size(Intersection(Group(b),Group(c)))<>1
     then continue;fi;
     if Group(a,b,c)<>G then continue;fi;triples:=triples+1;
     if Intersection(H,K)<>Group(b) then continue;fi;poly:=poly+1;
     rev:=GroupHomomorphismByImages(G,G,[a,b,c],[a^-1,a^2*b,c]);
     Need(rev<>fail and IsBijective(rev) and
     ForAll([a,b,c],x->Image(rev,Image(rev,x))=x),"every small polytopal
     triple has actual reversal");nonchiral:=nonchiral+1;
    od;
   od;
  od;
  Need(nonchiral=poly,"complete exclusion");
  Print("SMALL_ROTATIONS ",Size(G)," AUT ",Size(au)," PAIRS ",pairs,"
  GENERATING_TRIPLES ",triples," POLYTOPAL ",poly," CHIRAL0\n");
 od;
 Print("SMALL_ROTATION_EXCLUSIONS_PASS\n");
end;
Run();QUIT;
```

psl2_square_bound_check.py

```
#!/usr/bin/env python3
"""Exact integer verification of all finite Weil-bound exceptions in
Conjecture 6.1.

No third-party packages, floating point, probabilistic primality tests,
or external
factorization tables are used. This checks the numerical reduction; the
polynomial
```

```
and character-sum arguments are in psl2_square_proof.md. Separate GAP
files certify
the primitive finite-field witnesses for the cases where this bound is
inconclusive.
"""

from math import gcd, prod


def factor(n):
    out = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            out[d] = out.get(d, 0) + 1
            n //= d
        d = 3 if d == 2 else d + 2
    if n > 1:
        out[n] = out.get(n, 0) + 1
    return out


def odd_prime_power(n):
    fs = factor(n)
    if len(fs) != 1:
        return None
    p, g = next(iter(fs.items()))
    return (p, g) if p % 2 and g % 2 else None


def check_bound(q, a, require_nonzero=False):
    fs = factor(q - 1)
    assert prod(p**e for p, e in fs.items()) == q - 1
    w = 2 ** len(fs)
    # In the nested case up to four primitive roots of the quartic are
    removed.
    # Since 1/theta(q-1) <= W(q-1), the sufficient strengthened
    inequality is
    # q-1 > 4 W(q-1) (a sqrt(q)+1). Non-nested norms have no roots in
    F_q.
    left = q - 1 - (4 * w if require_nonzero else 0)
    return left > 0 and left**2 > 16 * a * a * w * w * q, fs, w


def main():
    # W(n) <= [64 / 30030^(1/4)] n^(1/4) < 5 n^(1/4).
    assert prod([2, 3, 5, 7, 11, 13]) == 30030
    assert 64**4 < 5**4 * 30030

    # For every odd b>=23, 3^b > [21(b-2)]^4:
    # establish b=23 and a lower bound for each b -> b+2 ratio.
    assert 3**23 > (21 * 21) ** 4
    assert 9 * 21**4 > 23**4

    nons_nested_failures = []
    tested = []
    for b in range(5, 23, 2):
        cap = (21 * (b - 2)) ** 4
```

```
    base = 3
    while base**b <= cap:
        pp = odd_prime_power(base)
        if pp:
            p, g = pp
            for a in range(3, b, 2):
                if gcd(a, b) != 1:
                    continue
                passed, fs, w = check_bound(base**b, a)
                record = (base, p, g, a, b, fs, w)
                tested.append(record)
                if not passed:
                    nons_nested_failures.append((base, a, b))
        base += 2
    # Exhibits the end of this finite base range explicitly.
    print(f"relative b={b}: all odd bases >= {base} satisfy the
    uniform bound")

expected_nonnested = [
    (3, 3, 5), (5, 3, 5),
    (3, 3, 7), (3, 5, 7),
    (3, 5, 9), (3, 7, 9),
]
assert nons_nested_failures == expected_nonnested
print("NONNESTED_EXCEPTIONS", nons_nested_failures)
print("NONNESTED_TESTED", len(tested))
print("COMPOSITE_BASES_TESTED", sorted({r[0] for r in tested if r[2]
> 1}))

cap = 21**4
nested_fields = []
nested_failures = []
p = 3
while p**3 <= cap:
    if factor(p) == {p: 1}:
        exponent = 3
        while p**exponent <= cap:
            q = p**exponent
            passed, fs, w = check_bound(q, 1, require_nonzero=True)
            nested_fields.append((p, exponent, fs, w))
            if not passed:
                # Every allowed first exponent divides the second
                exponent.
                for first_exponent in range(3, exponent + 1, 2):
                    if exponent % first_exponent == 0:
                        nested_failures.append((p, first_exponent,
                        exponent))
            exponent += 2
    p += 2
expected_nested = [
    (3, 3, 3), (3, 5, 5),
    (5, 3, 3), (7, 3, 3), (11, 3, 3),
]
assert nested_failures == expected_nested
assert len(nested_fields) == 23
print("NESTED_EXCEPTIONS", nested_failures)
print("NESTED_FIELDS_TESTED", len(nested_fields))
print("ALL_EXACT_INTEGER_CHECKS_PASSED")
```

```
if __name__ == "__main__":
    main()
```

verify_square_witnesses.py

```
"""Independent standard-library verification of primitive square
witnesses.

Finite-field arithmetic is implemented directly in polynomial quotients.
No GAP/Sage/SymPy output is assumed correct. A quotient generator of
order
p**degree-1 certifies that all its nonzero elements are units, hence that
the quotient is a field. Exact exponent tests certify this order.
"""
from pathlib import Path
import hashlib
import json
from math import gcd
import time


def prime_factors(n):
    factors = []
    d = 2
    while d * d <= n:
        if n % d == 0:
            factors.append(d)
            while n % d == 0:
                n //= d
        d += 1
    if n > 1:
        factors.append(n)
    return factors


class Quotient:
    def __init__(self, p, modulus):
        assert p > 2 and prime_factors(p) == [p]
        assert modulus[-1] == 1
        self.p, self.modulus = p, modulus
        self.degree = len(modulus) - 1
        self.q = p ** self.degree
        self.zero = (0,) * self.degree
        self.one = self.scalar(1)
        self.x = (0, 1) + (0,) * (self.degree - 2)

    def scalar(self, n):
        return (n % self.p,) + (0,) * (self.degree - 1)

    def add(self, x, y):
        return tuple((a + b) % self.p for a, b in zip(x, y))

    def neg(self, x):
        return tuple(-a % self.p for a in x)

    def sub(self, x, y):
```

```
        return self.add(x, self.neg(y))

    def mul(self, x, y):
        n, p = self.degree, self.p
        out = [0] * (2 * n - 1)
        for i, a in enumerate(x):
            for j, b in enumerate(y):
                out[i + j] += a * b
        for k in range(2 * n - 2, n - 1, -1):
            coefficient = out[k] % p
            for j in range(n):
                out[k - n + j] -= coefficient * self.modulus[j]
        return tuple(a % p for a in out[:n])

    def power(self, x, n):
        assert n >= 0
        value = self.one
        while n:
            if n & 1:
                value = self.mul(value, x)
            x = self.mul(x, x)
            n >>= 1
        return value

    def inverse(self, x):
        assert x != self.zero
        value = self.power(x, self.q - 2)
        assert self.mul(x, value) == self.one
        return value

    def primitive(self, x):
        order = self.q - 1
        assert self.power(x, order) == self.one
        for r in prime_factors(order):
            assert self.power(x, order // r) != self.one

    def nonzero_square(self, x):
        assert x != self.zero
        assert self.power(x, (self.q - 1) // 2) == self.one


def polynomial_mul(F, a, b):
    result = [F.zero] * (len(a) + len(b) - 1)
    for i, x in enumerate(a):
        for j, y in enumerate(b):
            result[i+j] = F.add(result[i+j], F.mul(x, y))
    return result


def norm_quartic(F, j):
    u = F.add(j, F.inverse(j))
    polynomial = [F.one]
    for i in range(F.degree):
        ui = F.power(u, F.p ** i)
        c = F.mul(ui, ui)
        edge = F.sub(c, F.scalar(4))
        middle = F.neg(F.add(F.mul(F.scalar(2), c), F.scalar(8)))
        polynomial = polynomial_mul(F, polynomial,
```

```
                                    [edge, F.zero, middle, F.zero, edge])
    assert all(all(c == 0 for c in coefficient[1:]) for coefficient in
    polynomial)
    return [coefficient[0] for coefficient in polynomial]


def evaluate(F, coefficients, x):
    result = F.zero
    for c in reversed(coefficients):
        result = F.add(F.mul(result, x), F.scalar(c))
    return result


def check_coprime(record):
    p, a, b = record['p'], record['a'], record['b']
    assert 1 < a < b and a % 2 == b % 2 == 1 and gcd(a, b) == 1
    A = Quotient(p, record['j1_minpoly'])
    B = Quotient(p, record['j2_minpoly'])
    assert (A.degree, B.degree) == (a, b)
    A.primitive(A.x)
    B.primitive(B.x)
    polynomial = norm_quartic(A, A.x)
    assert polynomial == record['normpoly']
    value = evaluate(B, polynomial, B.x)
    B.nonzero_square(value)
    return {'kind': 'coprime', 'p': p, 'a': a, 'b': b,
            'norm_value_coefficients': value, 'passed': True}


def check_nested(record):
    p, e = record['p'], record['e']
    assert e > 1 and e % 2 == 1
    F = Quotient(p, record['j1_minpoly'])
    assert F.degree == e
    F.primitive(F.x)
    j1 = F.power(F.x, record['j1_exponent'])
    j2 = F.power(F.x, record['j2_exponent'])
    F.primitive(j1)
    F.primitive(j2)
    u = F.add(j1, F.inverse(j1))
    v = F.add(j2, F.inverse(j2))
    u2, v2 = F.mul(u, u), F.mul(v, v)
    omega = F.sub(F.mul(u2, v2), F.mul(F.scalar(4), F.add(u2, v2)))
    F.nonzero_square(omega)
    return {'kind': 'nested', 'p': p, 'e': e,
            'omega_coefficients': omega, 'passed': True}


def main():
    root = Path(__file__).resolve().parent
    data_path = root / 'psl2_square_witnesses.json'
    data = json.loads(data_path.read_text())
    expected_coprime = {(3, 3, 5), (5, 3, 5), (3, 3, 7),
                        (3, 5, 7), (3, 5, 9), (3, 7, 9)}
    expected_nested = {(3, 3), (3, 5), (5, 3), (7, 3), (11, 3)}
    assert {(r['p'], r['a'], r['b']) for r in data if r['kind'] ==
    'coprime'} == expected_coprime
    assert {(r['p'], r['e']) for r in data if r['kind'] == 'nested'} ==
    expected_nested
```

```
    started = time.monotonic()
    results = [check_coprime(r) if r['kind'] == 'coprime' else
    check_nested(r)
               for r in data]
    assert len(results) == 11
    report = {'verifier_sha256':
    hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
              'data_sha256':
              hashlib.sha256(data_path.read_bytes()).hexdigest(),
              'elapsed_seconds': time.monotonic() - started, 'results':
              results,
              'all_passed': True}
    (root / 'psl2_square_witnesses_independent.json').write_text(
        json.dumps(report, indent=2) + '\n')
    print(json.dumps(report, indent=2))


if __name__ == '__main__':
    main()
```

psl2_square_witnesses.json


```
[
  {
    "kind": "coprime",
    "p": 3,
    "a": 3,
    "b": 5,
    "j1_minpoly": [
      1,
      2,
      0,
      1
    ],
    "j2_minpoly": [
      1,
      0,
      1,
      2,
      0,
      1
    ],
    "normpoly": [
      1,
      0,
      1,
      0,
      2,
      0,
      0,
      0,
      2,
      0,
      1,
      0,
      1
    ],
    "j1_exponent": 1,
```

```
  "j2_exponent": 7
},
{
  "kind": "coprime",
  "p": 5,
  "a": 3,
  "b": 5,
  "j1_minpoly": [
    3,
    3,
    0,
    1
  ],
  "j2_minpoly": [
    3,
    4,
    0,
    0,
    0,
    1
  ],
  "normpoly": [
    1,
    0,
    1,
    0,
    1,
    0,
    3,
    0,
    1,
    0,
    1,
    0,
    1
  ],
  "j1_exponent": 1,
  "j2_exponent": 1
},
{
  "kind": "coprime",
  "p": 3,
  "a": 3,
  "b": 7,
  "j1_minpoly": [
    1,
    2,
    0,
    1
  ],
  "j2_minpoly": [
    1,
    0,
    2,
    0,
    0,
    0,
    0,
```

```
      1
    ],
    "normpoly": [
      1,
      0,
      1,
      0,
      2,
      0,
      0,
      0,
      2,
      0,
      1,
      0,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 1
  },
  {
    "kind": "coprime",
    "p": 3,
    "a": 5,
    "b": 7,
    "j1_minpoly": [
      1,
      2,
      0,
      0,
      0,
      1
    ],
    "j2_minpoly": [
      1,
      0,
      2,
      0,
      0,
      0,
      0,
      1
    ],
    "normpoly": [
      1,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      2,
      0,
      2,
      0,
      2,
      0,
```

```
      0,
      0,
      0,
      0,
      0,
      0,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 1
  },
  {
    "kind": "coprime",
    "p": 3,
    "a": 5,
    "b": 9,
    "j1_minpoly": [
      1,
      2,
      0,
      0,
      0,
      1
    ],
    "j2_minpoly": [
      1,
      1,
      0,
      2,
      1,
      0,
      1,
      0,
      0,
      1
    ],
    "normpoly": [
      1,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      2,
      0,
      2,
      0,
      2,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      1
```

```
    ],
    "j1_exponent": 1,
    "j2_exponent": 5
  },
  {
    "kind": "coprime",
    "p": 3,
    "a": 7,
    "b": 9,
    "j1_minpoly": [
      1,
      0,
      2,
      0,
      0,
      0,
      0,
      1
    ],
    "j2_minpoly": [
      1,
      1,
      2,
      2,
      0,
      0,
      0,
      0,
      0,
      1
    ],
    "normpoly": [
      1,
      0,
      2,
      0,
      2,
      0,
      1,
      0,
      0,
      0,
      2,
      0,
      0,
      0,
      1,
      0,
      0,
      0,
      2,
      0,
      0,
      0,
      1,
      0,
      2,
      0,
```

```
      2,
      0,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 1
  },
  {
    "kind": "nested",
    "p": 3,
    "e": 3,
    "j1_minpoly": [
      1,
      2,
      0,
      1
    ],
    "j2_minpoly": [
      1,
      2,
      0,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 1
  },
  {
    "kind": "nested",
    "p": 3,
    "e": 5,
    "j1_minpoly": [
      1,
      2,
      0,
      0,
      0,
      1
    ],
    "j2_minpoly": [
      1,
      2,
      0,
      0,
      0,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 3
  },
  {
    "kind": "nested",
    "p": 5,
    "e": 3,
    "j1_minpoly": [
      3,
      3,
      0,
      1
```

```
    ],
    "j2_minpoly": [
      3,
      3,
      0,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 1
  },
  {
    "kind": "nested",
    "p": 7,
    "e": 3,
    "j1_minpoly": [
      4,
      0,
      6,
      1
    ],
    "j2_minpoly": [
      4,
      0,
      6,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 1
  },
  {
    "kind": "nested",
    "p": 11,
    "e": 3,
    "j1_minpoly": [
      9,
      2,
      0,
      1
    ],
    "j2_minpoly": [
      3,
      9,
      5,
      1
    ],
    "j1_exponent": 1,
    "j2_exponent": 3
  }
]
```